

Note: Improving LLM learning via Helper Models

Sharoon Austin

sharoona97@outlook.com

Abstract

Scaling up large language models (LLMs) remains a persistent challenge. We introduce a new mechanism, the *helper model*, in which a smaller pretrained model learns simple, complementary features from the training data and supplies them to a larger student model through cross-attention. In our implementation, the helper model uses sliding-window attention to capture short-range language structure, whereas the student model uses only dilated long-range attention. The two attention mechanisms therefore divide local and long-range processing between separate streams, with the helper representations supplying the local information omitted by the student’s attention pattern. Experiments with a basic LLM architecture show that a pretrained helper model accelerates loss reduction during the initial stages of training relative to an otherwise identical model with an untrained helper. As training progresses, the learning curves converge, consistent with the initially untrained helper gradually learning useful local features. These results suggest that separately pretrained helper models can improve the early-stage training efficiency of LLMs with complementary attention mechanisms.

1 Introduction

Recent work on Distillation from Weak Teachers (DWT) has demonstrated that smaller or weaker models can nevertheless provide useful training signals to larger student models [1, 2]. These approaches typically transfer information by modifying the student model’s loss function so that its output distribution is guided by predictions from the weak teacher. More generally, these results suggest that knowledge learned by a smaller model may remain useful even when that model has substantially less capacity than the model being trained.

In this work, we explore an architectural alternative to loss-based knowledge transfer [3]. We introduce the *helper model*: a smaller pretrained language model designed to learn simple, short-range patterns in language through sliding-window attention [4]. The larger student model, by contrast, uses dilated attention to process long-range dependencies. This construction assigns complementary roles to the two models: the helper captures local linguistic structure, while the student organizes information over longer distances. After pretraining, the helper model is incorporated directly into the student architecture through a one-way cross-attention mechanism. The student representations provide the queries, while the hidden representations of the helper provide the keys and values. This allows the student to selectively retrieve locally processed features during both training and inference. Unlike conventional distillation [3], the helper does not supervise the student through an additional output-level loss. Instead, its internal representations become an explicit computational component of the combined model, and the complete architecture is trained using the student’s ordinary next-token prediction objective.

The central hypothesis is that a pretrained helper can supply useful local language structure during the early stages of training, reducing the number of training updates required for the combined model to learn basic short-range statistics. The student can then devote its specialized long-range attention mechanism to modeling dependencies that cannot be captured within the helper’s restricted context window. We investigate whether this division of labor improves the loss trajectory relative to an otherwise identical architecture whose helper model is initialized without pretraining.

2 Background

In this section, we review the standard Transformer-based architecture used in LLMs [5]. An LLM is trained by minimizing the next-token prediction objective

$$\mathcal{L} = -\frac{1}{L-1} \sum_{k=1}^{L-1} \log p_{\theta}(t_{k+1}|t_{\leq k}), \quad (1)$$

where t_k denotes the token at position k , and L is the length of the token sequence. As illustrated in Fig. 1, the model consists of a sequence of Transformer blocks [5]. Given a token sequence $[t_1, \dots, t_L]$ of length L , the model first produces the corresponding vector embeddings $x = [x_1, \dots, x_L]$ using a module with learnable weights. Each embedding x_i has dimension d_{model} and is updated through a sequence of computational blocks called Transformer blocks.

We now describe the operation of an individual Transformer block. The block receives the sequence of vectors $x = [x_1, \dots, x_L]$ and applies a multi-head self-attention (MHSA) mechanism [5]. For each vector x_i , the first residual update is

$$y_i = x_i + \text{MHSA}(\text{Norm}(x))_i, \quad (2)$$

where each token position attends to itself and all preceding tokens in the sequence, and $\text{Norm}(x)$ denotes the normalization operation on all x_i in x . Each MHSA(.) block contains n_{head} triplets of linear modules with learnable weights (W_Q, W_K, W_V). The query and key modules, represented by W_Q and W_K , respectively,

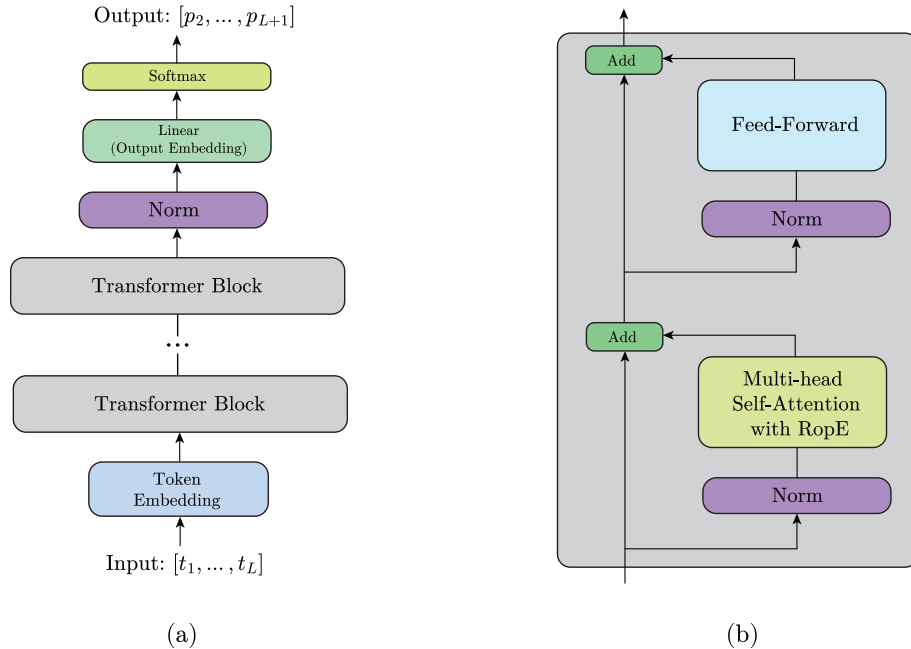


Figure 1: In (a), we show a schematic of the basic LLM architecture [6]. In (b), we show a schematic of a Transformer block.

have output dimension d_k , while W_V is the weight matrix associated with the value module. Each pair (W_Q, W_K) is used to calculate the attention scores between tokens. The query and key vectors are transformed

using the RoPE transformation R_{p_i} [7], where p_i is the position of token t_i . The resulting attention matrix is

$$S = \text{softmax}(\tilde{S} + M), \quad (3)$$

$$\tilde{S}_{ij} = q_i \cdot k_j / \sqrt{d_k}, \quad (4)$$

$$q_i = R_{p_i} W_Q(x_i), \quad k_j = R_{p_j} W_K(x_j), \quad i, j \in \{1, 2, \dots, L\}, \quad (5)$$

where $M_{ij} = -\infty$ for $p_j > p_i$ and zero otherwise. For each attention head with the triplet (W_Q, W_K, W_V) , the corresponding output for token t_i is $o_i = \sum_j S_{ij} W_V(x_j)$. The outputs from all attention heads are then concatenated and projected using a linear module with a learnable weight matrix W_O :

$$O_i = W_O \left(\text{concat}(o_i^{(1)}, \dots, o_i^{(n_{\text{heads}})}) \right), \quad (6)$$

where $\text{concat}(o_i^{(1)}, \dots, o_i^{(n_{\text{heads}})})$ denotes the concatenation of the outputs o_i from all attention heads. The linear module represented by W_O has output dimension d_{model} . Following the MHSA residual update in Eq. (2), the resulting vector is processed by a feed-forward network (FFN) through a second residual update:

$$z_i = y_i + \text{FFN}(\text{Norm}(y_i)), \quad (7)$$

where $\text{FFN}(\cdot)$ denotes the action of the feed-forward network. The resulting vector z_i has dimension d_{model} , completing the operation of a single Transformer block.

After applying n_{layers} Transformer blocks, each x_i is normalized and projected by a linear module with learnable weights W_F to a vector of dimension V , where V is the vocabulary size. The softmax function is then applied to $W_F(x_i)$ to produce a probability distribution $p_{i+1}(v)$ over the vocabulary elements v , representing the model's prediction for the token at position $(i + 1)$.

Throughout this work, the term *token embedding* refers to the vector representation associated with a token at a given stage of the model. The initial token embeddings are produced by the linear embedding module, while the embeddings at subsequent layers are contextualized representations obtained after applying Transformer blocks. Accordingly, x_i and h_i denote token embeddings at any depth of the student and helper models, respectively.

3 Training with a Helper Model

Our student and helper models are based on the Transformer architecture discussed previously. The helper model gathers simpler patterns in language through sliding-window attention (SWA) [4]: rather than allowing each query to attend to all preceding tokens in the input sequence, the query only attends to at most l_w preceding tokens. For token sequences of length L , this reduces the complexity of the attention mechanism from $O(L^2 d_{\text{model}})$ in the standard case to $O(L l_w d_{\text{model}})$. The intuition behind this construction is that the smaller helper model learns simpler phrase-level patterns of language which help the composite model accelerate its learning in the initial stages of training. For the student model, we employ dilated attention, whereby each token attends only to preceding tokens at regularly spaced intervals determined by a fixed stride length [4]. This reduces the complexity of the attention mechanism from $O(L^2 d_{\text{model}})$ to $O(L^2 d_{\text{model}} / l_s)$, where l_s is the stride length of the attention pattern.

We next describe how the pretrained helper model is coupled to the student model. The idea is to allow the student model to extract relevant information from the helper's token embeddings.

3.1 Cross-attention mechanism

We now describe the multi-head cross-attention (MHCA) mechanism that couples the helper model to the student model. Let $h_i \in \mathbb{R}^{d_h}$ denote the token embedding produced by the helper model for the token t_i ,

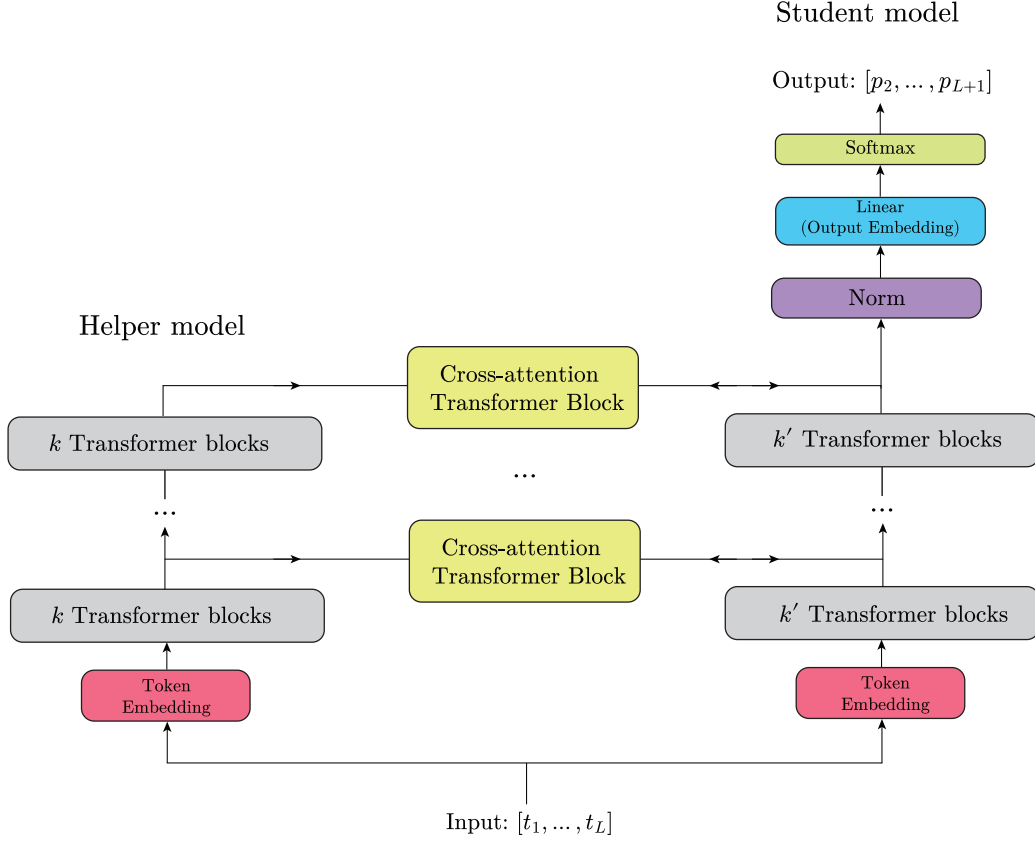


Figure 2: A schematic of the architecture with the helper model is shown. During training, the helper model provides its hidden representation to the student model via cross-attention transformer blocks. The student model incorporates the outputs from the cross-attention transformer blocks. The cross-attention block updates the student representation but does not feed an updated representation back into the helper stream.

and let $x_i \in \mathbb{R}^{d_s}$ denote the corresponding token embedding in the student model. The helper and student dimensions d_h and d_s need not be equal, and generally, $d_h \ll d_s$. In the construction considered here, information flows from the helper stream to the student stream, with no updated representation fed back into the helper stream by the cross-attention block.

As shown in Fig. 2, for each cross-attention head, the student model’s token embedding x_i supplies the query, whereas the helper model’s corresponding token embedding h_j supplies the keys and values, i.e., $q_i = R_{p_i} \tilde{W}_Q x_i$, $k_j = R_{p_j} \tilde{W}_K h_j$, $v_j = \tilde{W}_V h_j$. Here, R_{p_i} is the RoPE transformation encoding the position p_i of token t_i . Thus, the learned key and value projections translate the helper representation into the feature space used by the student. Using Eqs. (3)–(5), we compute the corresponding attention matrix $S^{(c)}$. The output from a single cross-attention head is $\tilde{o}_i = \sum_j S_{ij}^{(c)} \tilde{W}_V(h_j)$. The causal mask M is defined as in Eq. (3). Each $\tilde{o}_i$ is then processed as in Eq. (6) to obtain the output $\text{MHCA}(\cdot, \cdot)_i$. The cross-attention output is inserted into the student stream through a residual update,

$$y_i = x_i + \alpha \text{MHCA}(\text{Norm}(x), \text{Norm}(h))_i, \quad (8)$$

where α controls the strength of the helper contribution, $x = [x_1, \dots, x_L]$, and $h = [h_1, \dots, h_L]$. The choice $\alpha = 1$ gives the cross-attention output full residual weight, while a small fixed or learnable value makes the initial helper contribution perturbative. The complete cross-attention Transformer block then applies a

feed-forward network:

$$z_i = y_i + \text{FFN}(\text{Norm}(y_i)) . \quad (9)$$

This one-way coupling preserves the intended division of labor: the helper stream constructs local language features, while the student stream retrieves and integrates those features with its longer-range representation.

4 Results

We evaluate the proposed helper-model architecture using the FineWeb-Edu dataset [8]. We compare two versions of the same combined architecture. In the first, the helper model is randomly initialized and trained jointly with the student model. In the second, the helper model is first pretrained on the Cosmopedia dataset [9] and subsequently incorporated into the combined architecture for joint training with the student model on the FineWeb-Edu dataset. In the second case, the helper model is initialized using weights obtained from prior training on the same language-modeling objective in Eq. (1) and is subsequently allowed to continue adapting during the training of the combined model. Apart from the initialization of the helper model, the two cases use the same student architecture, cross-attention mechanism, and training procedure.

Fig. 3 shows the training-loss curves as a function of the number of training batches. The model with the pretrained helper exhibits a lower loss during the initial stages of training, indicating that the local representations learned by the helper provide an immediate advantage to the student model. As training proceeds, however, the difference between the two curves decreases. The convergence of the loss curves is consistent with the initially untrained helper gradually learning useful short-range features through gradients propagated from the student’s next-token objective.

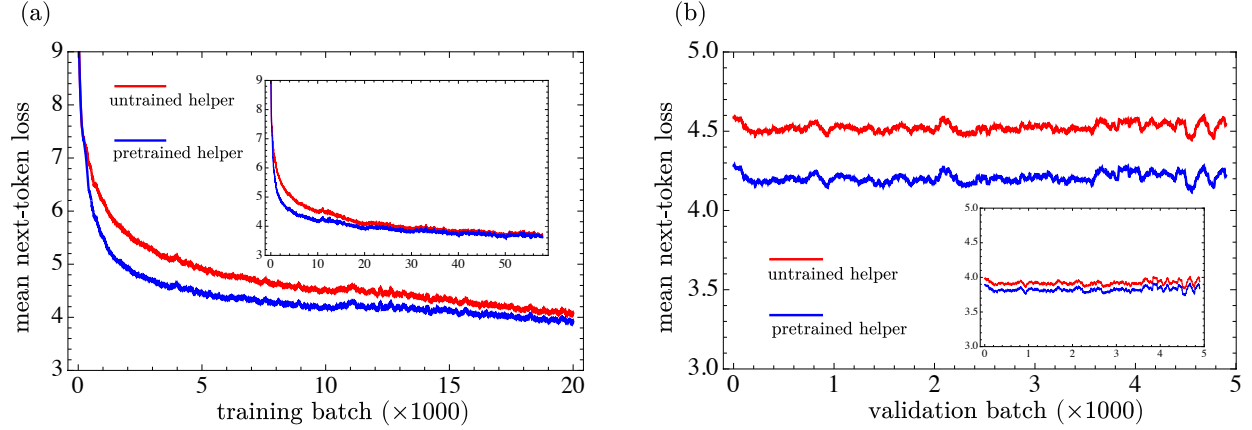


Figure 3: We show the mean next-token loss from Eq. (1) for the composite models with both untrained and pretrained helper models. We have $d_s = 1024$ for the student model and $d_h = 512$ for the helper model. The student model contains eight Transformer layers with eight attention heads per layer, while the helper model contains six Transformer layers with eight attention heads per layer. The architecture contains six cross-attention Transformer blocks, each with eight attention heads. The student model’s attention heads have dilated attention with stride length $l_s = 8$ [4], while the helper model’s attention heads have sliding-window attention [4] with context window $l_w = 15$. The token sequence length is $L = 256$, and each batch contains 20 token sequences. The helper model is pretrained using the Cosmopedia dataset [9]. Training and validation are performed using the FineWeb-Edu dataset [8]. For the cross-attention mechanism, $\alpha = 1$. Training is performed using a learning rate between 10^{-5} and 10^{-4} , with linear warm-up followed by cosine decay. In (a), we show the training loss, and in (b) we show the validation loss. The inset in (a) shows the training loss curves for a larger number of training batches. For (b), the main plot shows the validation loss for models trained on 10,000 training batches, and the inset shows models trained on 30,000 training batches. All results are average-pooled using a window size of 100 and a stride value of 1.

These results suggest that helper-model pretraining primarily provides an optimization advantage during the early stages of training. As the difference between the two models decreases, subsequent loss reduction appears to be governed increasingly by the student model’s ability to learn long-range dependencies.

5 Conclusion

In this work, we introduced the *helper model*, an architectural mechanism for transferring representations learned by a smaller pretrained language model into a larger student model. The helper model uses sliding-window attention to capture short-range language structure, while the student model uses dilated attention to process longer-range dependencies. One-way cross-attention blocks allow the student to retrieve the helper’s local representations while preserving the complementary roles of the two streams. Unlike conventional knowledge-distillation methods, the proposed mechanism transfers information through intermediate hidden representations rather than through an additional output-level loss.

Our numerical experiments on the FineWeb-Edu dataset show that pretraining the helper model improves the initial loss trajectory of the combined model relative to an otherwise identical architecture with a randomly initialized helper. A similar improvement is observed in the validation loss. However, this advantage decreases over time, consistent with the randomly initialized helper gradually learning useful local features through the student’s next-token objective. These findings provide preliminary evidence that separately pretrained specialist models can serve as reusable feature providers during the training of larger language models. We

anticipate that the helper-model framework can be extended to other complementary forms of specialization, thereby providing a modular approach to learning and organizing features across different levels of linguistic structure. Future work may investigate whether analogous helper models can support other capabilities, including reasoning.

References

- [1] Li Yuan, Francis E. H. Tay, Guilin Li, Tao Wang, and Jiashi Feng. Revisiting knowledge distillation via label smoothing regularization, 2021.
- [2] Hayeon Lee, Rui Hou, Jongpil Kim, Davis Liang, Sung Ju Hwang, and Alexander Min. A study on knowledge distillation from weak teacher for scaling up pre-trained language models, 2023.
- [3] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- [4] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers, 2019.
- [5] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.
- [6] Stanford CS336. Assignment 1: Basics — language modeling from scratch. <https://github.com/stanford-cs336/assignment1-basics>, 2025. CS336: Language Modeling From Scratch, Stanford University.
- [7] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2023.
- [8] Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The fineweb datasets: Decanting the web for the finest text data at scale, 2024.
- [9] Loubna Ben Allal, Anton Lozhkov, Guilherme Penedo, Thomas Wolf, and Leandro von Werra. Cosmo-pedia, February 2024.